

PROJETO - ECO CIDADES

Henrique Gheno

Descrição do Pipeline CI/CD

O projeto Eco Cidades emprega o GitHub Actions para automatizar integralmente os processos de Integração Contínua (CI) e Entrega Contínua (CD). Esta abordagem garante a qualidade do código, a execução de testes e a implantação eficiente da aplicação em ambientes de produção.

Ferramenta Utilizada

O GitHub Actions é a ferramenta central para a orquestração do pipeline. Ele permite a definição de workflows diretamente no repositório do código, facilitando a automação de tarefas como build, teste e deploy.

Etapas e Lógica do Pipeline

O pipeline é composto por dois workflows principais:

1. Workflow de Integração Contínua (continuous_integration.yml)

Acionamento: Este workflow é ativado automaticamente a cada pull_request direcionado à branch develop.

Etapas:

- Git Checkout: Clona o código-fonte do repositório para o ambiente de execução do workflow.
- Setup Java SDK: Configura o ambiente de desenvolvimento Java, especificamente a versão 21 do JDK (Temurin), essencial para a compilação do projeto Spring Boot.
- Unit Tests: Executa os testes unitários definidos no projeto Maven (mvn test). Esta etapa é crucial para validar a lógica de negócio e garantir que novas alterações não introduzem regressões.
- Lógica: O objetivo principal deste workflow é garantir que todas as novas contribuições de código passem por uma validação rigorosa antes de serem mescladas na branch principal de desenvolvimento, mantendo a estabilidade e a qualidade do código base.

2. Workflow de Entrega Contínua (continuous-delivery.yml)

Acionamento: Este workflow é disparado automaticamente a cada push para a branch develop.

Etapas:

- Checkout: Clona o repositório.
- Set up Docker Buildx: Configura o Docker Buildx, uma ferramenta que estende as capacidades de docker build, permitindo construções mais eficientes e multi-arquitetura.

- Log in to registry: Autentica-se no Docker Hub utilizando credenciais (DOCKERHUB_USERNAME, DOCKERHUB_TOKEN) armazenadas de forma segura como segredos do GitHub. Isso é necessário para enviar a imagem Docker construída.
- Build and push container image to registry: Constrói a imagem Docker da aplicação a partir do Dockerfile e a envia para o Docker Hub. A imagem é taggeada com o SHA do commit do GitHub, garantindo rastreabilidade.
- Deploy to Azure Web App: Realiza a implantação da imagem Docker recém-construída e enviada para um serviço de aplicação no Microsoft Azure. O deploy é direcionado ao Azure Web App ecocidades-api no slot de production, utilizando um perfil de publicação (AZURE_PROFILE) para autenticação e configuração.
- Lógica: Este workflow assegura que, uma vez que o código tenha passado pela integração contínua e seja mesclado na branch develop, uma nova versão containerizada da aplicação seja automaticamente construída, publicada e implantada no ambiente de produção, garantindo uma entrega rápida e confiável.

Docker: Arquitetura, Comandos e Imagem Criada

A aplicação Eco Cidades é projetada para ser executada em containers Docker, o que proporciona portabilidade, isolamento e escalabilidade.

Arquitetura de Containerização

O Dockerfile emprega uma estratégia de build multi-stage, que é uma prática recomendada para criar imagens Docker otimizadas. Esta abordagem divide o processo de construção em duas fases principais:

1. Fase de Build: Utiliza uma imagem base (maven:3.9.8-eclipse-temurin-21) que contém todas as ferramentas necessárias para compilar o projeto Java (Maven e JDK). Nesta fase, o código-fonte é copiado, as dependências são resolvidas e o arquivo .jar executável da aplicação é gerado.
2. Fase de Runtime: Utiliza uma imagem base minimalista (eclipse-temurin:21-jdk-alpine) que contém apenas o ambiente de execução Java (JRE) necessário. O arquivo .jar gerado na fase de build é copiado para esta imagem. Isso resulta em uma imagem final significativamente menor, mais segura e com menos superfície de ataque, ideal para ambientes de produção.

```
FROM maven:3.9.8-eclipse-temurin-21 AS build

RUN mkdir /opt/app

COPY . /opt/app

WORKDIR /opt/app

RUN mvn clean package

FROM eclipse-temurin:21-jdk-alpine

RUN mkdir /opt/app

COPY --from=build /opt/app/target/app.jar /opt/app/app.jar

WORKDIR /opt/app

ENV PROFILE=prd

EXPOSE 8080

ENTRYPOINT ["java", "-Dspring.profiles.active=${PROFILE}", "-jar", "app.jar"]
```

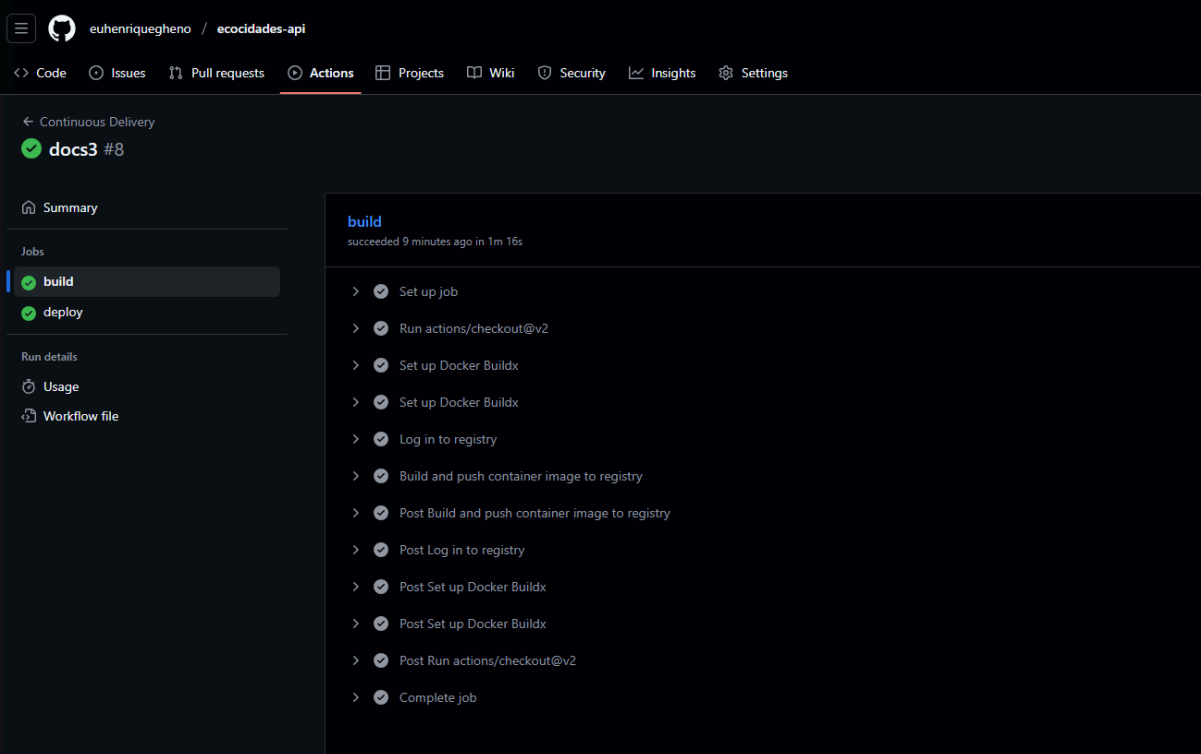
Comandos Docker Essenciais

- Construir a imagem: `docker build -t ecocidades-api .`
- Este comando constrói a imagem Docker a partir do Dockerfile no diretório atual e a taggeia como `ecocidades-api`.
- Executar o contêiner: `docker run -p 8080:8080 ecocidades-api`
- Inicia um contêiner a partir da imagem `ecocidades-api`, mapeando a porta 8080 do contêiner para a porta 8080 da máquina host, tornando a API acessível localmente.

Imagem Criada

A imagem Docker final é um pacote autocontido que inclui a aplicação Spring Boot e todas as suas dependências, pronta para ser executada em qualquer ambiente que suporte Docker. A tag da imagem no Docker Hub é `index.docker.io/${{ secrets.DOCKERHUB_USERNAME }}/ecocidades-api:${{ github.sha }}`, onde `${{ github.sha }}` garante uma versão única para cada build do pipeline de CD.

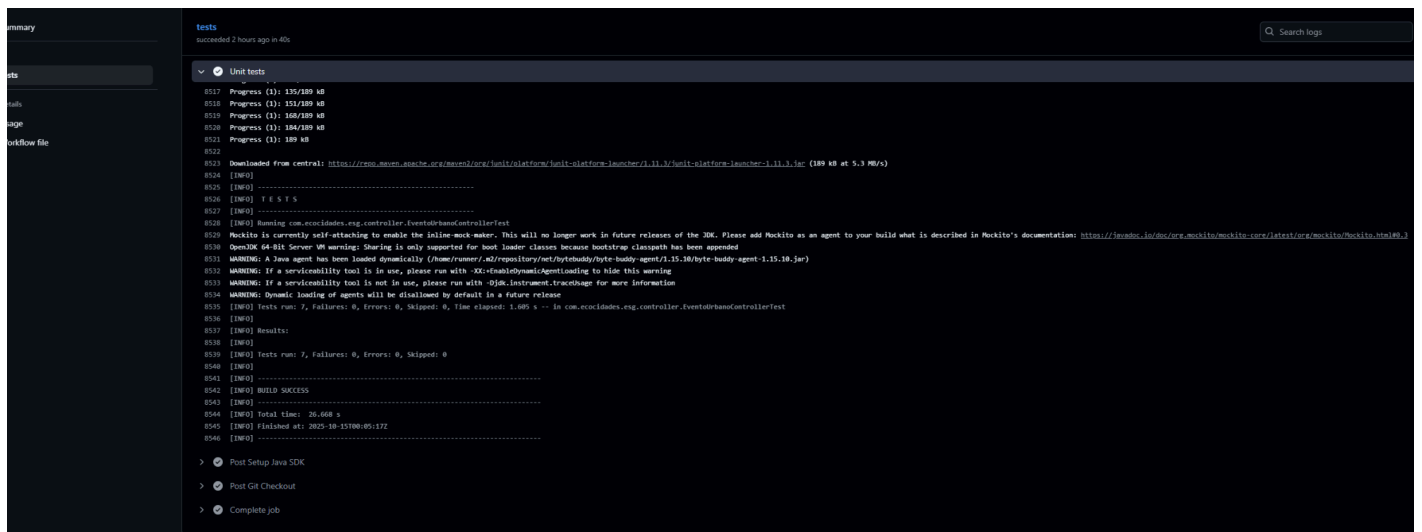
Pipeline Rodando



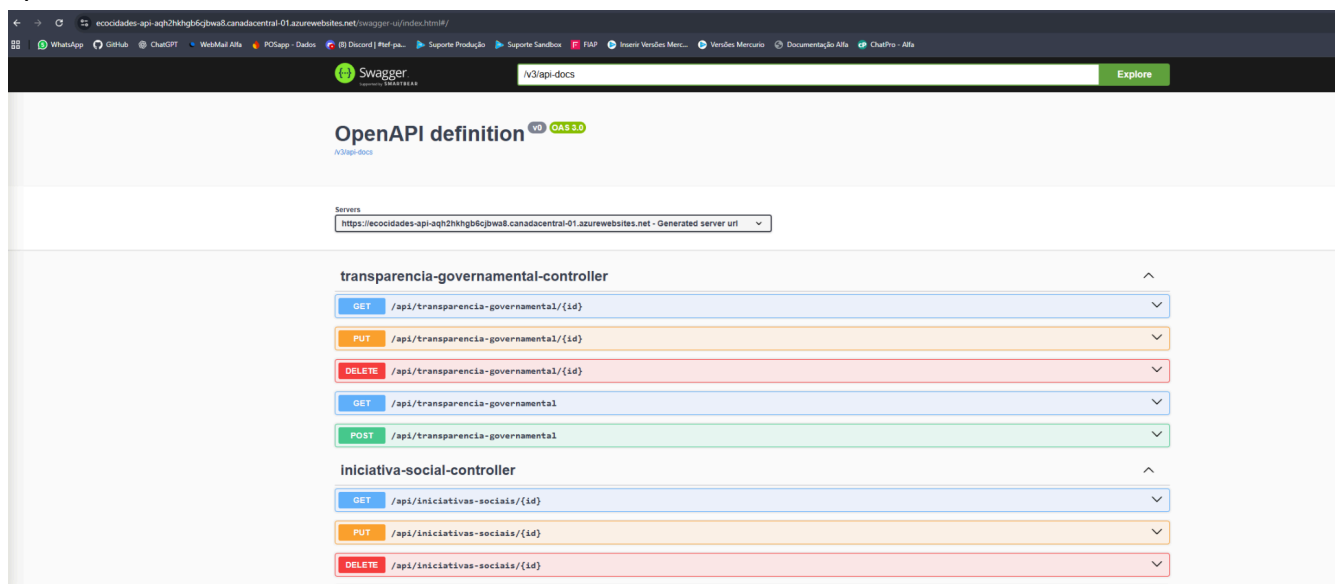
The screenshot displays the GitHub Actions interface for a repository named 'ecocidades-api' by user 'euenriquegheno'. The 'Actions' tab is selected, showing a workflow named 'docs3' with run #8. The 'build' job is highlighted in the 'Jobs' list on the left. The main panel shows the 'build' job details, indicating it 'succeeded 9 minutes ago in 1m 16s'. A list of 13 steps follows, all marked as successful with green checkmarks:

- > Set up job
- > Run actions/checkout@v2
- > Set up Docker Buildx
- > Set up Docker Buildx
- > Log in to registry
- > Build and push container image to registry
- > Post Build and push container image to registry
- > Post Log in to registry
- > Post Set up Docker Buildx
- > Post Set up Docker Buildx
- > Post Run actions/checkout@v2
- > Complete job

Unit test:



Api rodando:



Acesse a api (Swagger)

Checklist de Entrega

- Projeto compactado em .ZIP com estrutura organizada ✓
- Dockerfile funcional ✓
- docker-compose.yml ou arquivos Kubernetes ✓

- Pipeline com etapas de build, teste e deploy 
- README.md com instruções e prints 
- Documentação técnica com evidências (PDF ou PPT) 
- Deploy realizado nos ambientes staging e produção 